



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

Projekt i implementacja aplikacji webowej do transkrypcji nagrań
fortepianowych na nuty w formacie PDF z wykorzystaniem
zaawansowanych modeli AI

Michał JAGODA

Nr albumu: 300405

Kierunek: Informatyka

Specjalność: Grafika Komputerowa i Oprogramowanie

PROWADZĄCY PRACĘ

Dr hab. inż. Karolina Nurzyńska

KATEDRA Algorytmiki i Oprogramowania

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2024

Tytuł pracy

Projekt i implementacja aplikacji webowej do transkrypcji nagrań fortepianowych na nuty w formacie PDF z wykorzystaniem zaawansowanych modeli AI

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

Thesis title

Design and implementation of a web application for transcribing piano recordings into musical notes in PDF format using advanced AI models

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

Spis treści

1	Wstęp	1
2	Analiza tematu	3
2.1	Sformułowanie problemu	3
2.2	Osadzenie tematu w kontekście aktualnego stanu wiedzy	4
2.3	Studia literaturowe	5
3	Wymagania i narzędzia	9
3.1	Wymagania funkcjonalne	9
3.2	Wymagania нефункционалне	10
3.3	Wymagania sprzętowe i programowe	11
3.3.1	Wymagania dla użytkowników	11
3.3.2	Wymagania dla serwera	11
3.4	Sposób instalacji	12
3.4.1	Skrypty developerskie	12
3.5	Przypadki użycia	13
4	Specyfikacja zewnętrzna	15
4.1	Wymagania sprzętowe i programowe	15
4.1.1	Wymagania sprzętowe	15
4.1.2	Wymagania programowe	15
4.2	Sposób instalacji	15
4.3	Kategorie użytkowników	16
4.4	Sposób obsługi	16
4.5	Administracja systemem	17
4.6	Kwestie bezpieczeństwa	18
4.7	Przykład działania	18
4.8	Scenariusze korzystania z systemu	18
5	Specyfikacja wewnętrzna	19
5.1	Представление идеи	19
5.2	Architektura systemu	19

5.3	Opis struktur danych i organizacji bazy danych	20
5.4	Komponenty, moduły i biblioteki	20
5.5	Skrypty deweloperskie	21
5.6	Przegląd ważniejszych algorytmów	22
5.7	Szczegóły implementacji wybranych fragmentów	22
5.7.1	Implementacja modelu transkrypcji	22
5.7.2	Implementacja API GraphQL	23
5.8	Diagramy UML	24
6	Weryfikacja i walidacja	25
7	Podsumowanie i wnioski	27
	Bibliografia	29
	Spis skrótów i symboli	33
	Źródła	35
	Lista dodatkowych plików, uzupełniających tekst pracy	37
	Spis rysunków	39
	Spis tabel	41

Rozdział 1

Wstęp

- wprowadzenie w problem/zagadnienie
- osadzenie problemu w dziedzinie
- cel pracy
- zakres pracy
- zwięzła charakterystyka rozdziałów
- jednoznaczne określenie wkładu autora, w przypadku prac wieloosobowych – tabela z autorstwem poszczególnych elementów pracy

Rozdział 2

Analiza tematu

2.1 Sformułowanie problemu

Współczesna muzyka fortepianowa często występuje w formie cyfrowych nagrań audio, najczęściej w formacie MP3. Format ten, choć wygodny do odtwarzania, nie pozwala na łatwą edycję czy analizę struktury muzycznej. Muzycy, kompozytorzy, pedagodzy muzyczni oraz entuzjaści fortepianu potrzebują narzędzia, które umożliwi im konwersję plików audio do formatu MIDI, a następnie do czytelnych partytur w formacie PDF.

Problem, którym zajmuje się niniejsza praca, polega na stworzeniu efektywnego, dokładnego i przyjaznego dla użytkownika systemu, który automatyzuje proces transkrypcji muzyki fortepianowej. System ten, nazwany TuneScript, ma za zadanie przekształcać nagrania fortepianowe z formatu MP3 do symbolicznej reprezentacji MIDI, a następnie generować profesjonalnie wyglądające partytury w formacie PDF.

Kluczowe wyzwania, które należy rozwiązać w ramach tego projektu, obejmują:

- Precyzyjne rozpoznawanie wysokości dźwięków w nagraniach fortepianowych, z uwzględnieniem złożoności polifonii fortepianowej.
- Dokładne określanie czasu rozpoczęcia i zakończenia każdej nuty, co jest kluczowe dla zachowania rytmu i tempa utworu.
- Odwzorowanie dynamiki gry, która jest istotnym elementem ekspresji muzycznej na fortepianie.
- Efektywne przetwarzanie długich utworów muzycznych przy zachowaniu wysokiej jakości transkrypcji.
- Generowanie czytelnych i estetycznych partytur PDF, które będą użyteczne dla muzyków.

Rozwiązanie tego problemu ma potencjał znaczącego usprawnienia pracy muzyków, ułatwienia analizy muzycznej oraz wsparcia procesów edukacyjnych w dziedzinie muzyki fortepianowej.

2.2 Osadzenie tematu w kontekście aktualnego stanu wiedzy

Automatyczna transkrypcja muzyki (ATM, ang. *Automatic Music Transcription*) jest jednym z najbardziej złożonych problemów w dziedzinie przetwarzania sygnałów muzycznych (MIR, ang. *Music Information Retrieval*). W kontekście muzyki fortepianowej, ATM zyskuje szczególne znaczenie ze względu na popularność tego instrumentu oraz jego rolę w edukacji muzycznej i kompozycji.

Obecny stan wiedzy w dziedzinie ATM dla fortepianu charakteryzuje się następującymi aspektami:

1. **Wykorzystanie głębokiego uczenia:** Współczesne podejścia do ATM fortepianowego opierają się głównie na zastosowaniu głębokich sieci neuronowych. Modele takie jak konwolucyjne sieci neuronowe (CNN) i rekurencyjne sieci neuronowe (RNN) wykazały znaczącą skuteczność w rozpoznawaniu struktur muzycznych w spektrogramach audio [3].
2. **Modele hybrydowe:** Najnowsze badania skupiają się na modelach hybrydowych, łączących różne architektury sieci neuronowych. Na przykład, model "Onsets and Frames" [3] wykorzystuje osobne sieci do detekcji początków dźwięków i ich trwania, co znacząco poprawiło dokładność transkrypcji.
3. **Przetwarzanie w czasie rzeczywistym:** Rozwój technik przetwarzania sygnałów i optymalizacji algorytmów pozwala na coraz szybszą transkrypcję, zbliżającą się do przetwarzania w czasie rzeczywistym [5].
4. **Modelowanie akustyki fortepianu:** Zaawansowane modele uwzględniają specyfikę akustyki fortepianu, w tym zjawiska takie jak rezonans strun i wpływ pedałów na brzmienie [2].
5. **Transkrypcja ekspresji muzycznej:** Nowsze badania skupiają się nie tylko na rozpoznawaniu nut, ale także na odwzorowaniu dynamiki, artykulacji i innych aspektów ekspresji muzycznej [4].

Mimo znaczących postępów w dziedzinie ATM dla fortepianu, wciąż istnieją pewne ograniczenia i wyzwania:

- **Dokładność w muzyce polifonicznej:** Transkrypcja złożonych struktur polifonicznych, charakterystycznych dla zaawansowanej literatury fortepianowej, wciąż stanowi wyzwanie dla obecnych systemów [1].
- **Rozpoznawanie bardzo cichych dźwięków:** Detekcja nut granych bardzo cicho, szczególnie w kontekście głośniejszego akompaniamentu, pozostaje problematyczna [7].
- **Odróżnianie bliskich harmonicznie dźwięków:** W gęstych strukturach harmonicznych, rozróżnienie blisko brzmiących dźwięków może być trudne dla systemów ATM [1].
- **Interpretacja rytmiczna:** Precyzyjne odwzorowanie złożonych struktur rytmicznych, szczególnie w muzyce z rubato lub improwizowanej, stanowi wyzwanie dla obecnych systemów [2].
- **Wykorzystanie pedałów:** Uwzględnienie u użytkownika pedałów fortepianowych, które wpływają na rezonans strun i brzmienie, wymaga zaawansowanych technik analizy akustycznej [2].

Projekt TuneScript wpisuje się w ten kontekst, stawiając sobie za cel adresowanie wymienionych wyzwań poprzez integrację zaawansowanego modelu AI do transkrypcji z MP3 do MIDI z narzędziami do generowania partytur PDF. Takie kompleksowe podejście ma potencjał przezwyciężenia niektórych ograniczeń obecnych rozwiązań i dostarczenia użytkownikom narzędzia o wysokiej praktycznej wartości.

2.3 Studia literaturowe

W celu głębszego zrozumienia problematyki automatycznej transkrypcji muzyki fortepianowej oraz określenia aktualnego stanu wiedzy w tej dziedzinie, przeprowadzono szczegółowy przegląd literatury. Poniżej przedstawiono kluczowe prace i ich wkład w rozwój tej dziedziny:

1. **Kong et al. (2020)** [6] w pracy "High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times" prezentują nowatorskie podejście do transkrypcji fortepianowej, które stanowi podstawę modelu wykorzystywanego w projekcie TuneScript. Autorzy proponują metodę wysokiej rozdzielczości czasowej do transkrypcji fortepianowej, która uwzględnia nie tylko detekcję nut, ale także użycie pedałów. Kluczową innowacją jest zastosowanie regresji do precyzyjnego określania czasów początku (onset) i końca (offset) każdej nuty, co pozwala na dokładniejsze odwzorowanie rytmu i artykulacji. Model ten wykazuje znaczącą poprawę w

stosunku do wcześniejszych rozwiązań, szczególnie w zakresie precyzji czasowej i detekcji użycia pedałów.

2. **Benetos et al. (2013)** [1] w swojej pracy przeglądowej "Automatic music transcription: challenges and future directions" przedstawiają kompleksowe omówienie technik ATM. Autorzy omawiają główne wyzwania w dziedzinie, takie jak polifonia, różnorodność instrumentów i style wykonania. W kontekście muzyki fortepianowej, praca ta podkreśla złożoność problemu transkrypcji ze względu na szeroką gamę dynamiczną i bogactwo harmoniczne fortepianu.
3. **Hawthorne et al. (2018)** [3] w artykule "Onsets and frames: Dual-objective piano transcription" prezentują przełomowy model do transkrypcji fortepianowej. Ich podejście, nazwane "Onsets and Frames", wykorzystuje osobne sieci neuronowe do detekcji początków dźwięków i ich trwania. Model ten znacząco poprawił dokładność transkrypcji fortepianowej, osiągając wyniki bliskie ludzkiej dokładności w niektórych aspektach. Jest to jedno z najbardziej zaawansowanych rozwiązań w dziedzinie ATM dla fortepianu, które stanowiło inspirację dla późniejszych prac, w tym modelu Kong et al.
4. **Kelz et al. (2016)** [5] w pracy "On the Potential of Simple Framewise Approaches to Piano Transcription" badają możliwości prostszych podejść do transkrypcji fortepianowej, opierających się na analizie pojedynczych ramek czasowych. Autorzy wykazują, że nawet stosunkowo proste modele mogą osiągać konkurencyjne wyniki, co ma istotne implikacje dla projektowania systemów działających w czasie rzeczywistym.
5. **Cheng et al. (2016)** [2] w artykule "Attack/decay analysis of piano sounds using an exponential model" skupiają się na modelowaniu akustycznych właściwości dźwięków fortepianowych. Ich praca jest istotna dla zrozumienia, jak precyzyjnie analizować początkową fazę dźwięku (atak) oraz jego wygaszanie (decay), co jest kluczowe dla dokładnej transkrypcji dynamiki i artykulacji.
6. **Jeong et al. (2017)** [4] w pracy "Deep Learning for Music" prezentują szerokie zastosowanie technik głębokiego uczenia w różnych zadaniach związanych z przetwarzaniem muzyki, w tym w transkrypcji fortepianowej. Autorzy omawiają, jak różne architektury sieci neuronowych mogą być wykorzystane do modelowania różnych aspektów muzyki, od wysokości dźwięku po ekspresję.
7. **Sigtia et al. (2016)** [7] w pracy "An end-to-end neural network for polyphonic piano music transcription" prezentują kompleksowe podejście do transkrypcji muzyki fortepianowej wykorzystujące głębokie sieci neuronowe. Ich model jest w stanie

bezpośrednio przetwarzać surowy sygnał audio na notację muzyczną, co stanowi znaczący krok w kierunku w pełni automatycznej transkrypcji.

Powyższe prace stanowią fundament teoretyczny dla projektu TuneScript. Pokazują one ewolucję technik ATM dla fortepianu od tradycyjnych metod przetwarzania sygnałów do zaawansowanych modeli uczenia głębokiego. Jednocześnie, literatura ta wskazuje na wciąż istniejące wyzwania, szczególnie w zakresie transkrypcji muzyki polifonicznej, dokładnego odwzorowania dynamiki i ekspresji oraz generowania wysokiej jakości partytur.

Projekt TuneScript czerpie z tych badań, integrując najnowsze osiągnięcia w dziedzinie ATM z praktycznym aspektem generowania użytecznych partytur. Szczególnie istotna jest praca Kong et al. [6], która stanowi podstawę modelu transkrypcji wykorzystywanego w systemie. Model ten, dzięki swojej wysokiej rozdzielczości czasowej i uwzględnieniu pedałów, pozwala na bardziej precyzyjną transkrypcję niż wcześniejsze rozwiązania.

Warto zauważyć, że mimo znaczącego postępu w dziedzinie ATM dla fortepianu, wciąż istnieje przestrzeń do dalszych ulepszeń, szczególnie w zakresie dokładności transkrypcji bardzo złożonych utworów polifonicznych oraz wiernego odwzorowania subtelności ekspresji muzycznej. TuneScript stara się adresować te wyzwania, oferując kompleksowe rozwiązanie łączące zaawansowaną transkrypcję z praktycznym zastosowaniem w postaci generowania partytur.

Rozdział 3

Wymagania i narzędzia

3.1 Wymagania funkcjonalne

System TuneScript powinien spełniać następujące wymagania funkcjonalne:

1. Uwierzytelnianie i autoryzacja użytkowników:

- System musi zapewniać funkcjonalności rejestracji, logowania i wylogowywania użytkowników.
- Użytkownicy muszą mieć możliwość resetowania haseł za pomocą poczty elektronicznej.

2. Przesyłanie plików:

- System musi umożliwiać użytkownikom przesyłanie plików audio w formatach MP3 i WAV.
- System musi weryfikować format przesyłanych plików.

3. Przetwarzanie transkrypcji:

- System musi przetwarzać przesłane pliki audio w celu generowania transkrypcji fortepianowych.
- Transkrypcje powinny być zapisywane w formacie MIDI oraz generowane jako partytury PDF.

4. Zarządzanie transkrypcjami:

- Użytkownicy muszą mieć możliwość przeglądania, edytowania i usuwania swoich transkrypcji.
- Użytkownicy muszą mieć możliwość ustawiania widoczności swoich transkrypcji jako publicznych lub prywatnych.

5. Funkcjonalność wyszukiwania:

- System musi zapewniać możliwości wyszukiwania transkrypcji na podstawie tytułu, kompozytora i widoczności.

6. System oceniania:

- Użytkownicy muszą mieć możliwość oceniania transkrypcji.
- System musi obliczać i wyświetlać średnią ocenę dla każdej transkrypcji.

7. Zakładki:

- Użytkownicy muszą mieć możliwość dodawania transkrypcji do ulubionych.
- Użytkownicy muszą mieć możliwość przeglądania i zarządzania swoimi ulubionymi transkrypcjami.

3.2 Wymagania niefunkcjonalne

System TuneScript powinien spełniać następujące wymagania niefunkcjonalne:

1. Wydajność:

- System musi przetwarzać transkrypcje w rozsądnych ramach czasowych (np. mniej niż 5 minut dla 10-minutowego pliku audio).
- System powinien obsługiwać jednocześnie co najmniej 100 aktywnych użytkowników.

2. Użyteczność:

- Interfejs użytkownika musi być intuicyjny i łatwy w nawigacji.
- System musi dostarczać użytkownikom jasne instrukcje i informacje zwrotne.

3. Niezawodność:

- System musi być niezawodny, z dostępnością na poziomie 99,9
- System musi zapewniać mechanizmy automatycznego odzyskiwania po awariach.

4. Bezpieczeństwo:

- System musi chronić dane użytkowników za pomocą szyfrowania.
- System musi wykorzystywać bezpieczne mechanizmy uwierzytelniania, w tym dwuskładnikowe uwierzytelnianie.

5. Kompatybilność:

- System musi być kompatybilny z najnowszymi wersjami przeglądarek Chrome, Firefox, Safari i Edge.
- Interfejs użytkownika powinien być responsywny i działać na urządzeniach mobilnych.

6. Skalowalność:

- System musi być zdolny do obsługi rosnącej liczby użytkowników i transkrypcji bez znaczącego wpływu na wydajność.

7. Łatwość utrzymania:

- System musi być modułowy, umożliwiając łatwe aktualizacje i rozszerzenia.
- System musi posiadać kompletną dokumentację techniczną i użytkownika.

3.3 Wymagania sprzętowe i programowe

3.3.1 Wymagania dla użytkowników

- Urządzenie z dostępem do internetu (komputer, tablet lub smartfon)
- Nowoczesna przeglądarka internetowa (Chrome, Firefox, Safari lub Edge w najnowszej wersji)
- Minimum 4 GB RAM dla optymalnego działania aplikacji
- Stabilne połączenie internetowe o prędkości co najmniej 5 Mbps

3.3.2 Wymagania dla serwera

- Procesor: minimum 4 rdzenie, zalecane 8 rdzeni
- RAM: minimum 16 GB, zalecane 32 GB
- Dysk: minimum 100 GB przestrzeni dyskowej SSD
- System operacyjny: Linux (zalecany Ubuntu 20.04 LTS lub nowszy)
- Stabilne połączenie internetowe o wysokiej przepustowości

3.4 Sposób instalacji

Instalacja systemu TuneScript jest prosta i wymaga jedynie kilku kroków:

1. Sklonowanie repozytorium projektu:

```
git clone https://github.com/mj300405/TuneScript.git
```

2. Przejście do katalogu narzędzi deweloperskich projektu:

```
cd TuneScript/devtools
```

3. Uruchomienie Docker Compose, aby zbudować i uruchomić kontenery (wymagana wcześniejsza instalacja Dockera):

```
npm run start
```

4. Po zakończeniu procesu budowania i uruchamiania, aplikacja będzie dostępna w przeglądarce pod adresem:

```
http://localhost:3000
```

Uwaga: Przed uruchomieniem Docker Compose należy upewnić się, że Docker jest zainstalowany na systemie. W razie potrzeby można go pobrać i zainstalować ze strony oficjalnej: <https://www.docker.com/get-started>

3.4.1 Skrypty deweloperskie

System TuneScript zawiera zestaw skryptów ułatwiających proces developmentu i testowania. Skrypty te pozwalają na łatwe uruchamianie, zatrzymywanie i restartowanie aplikacji w różnych trybach, a także uruchamianie testów i formatowanie kodu. Szczegółowe informacje na temat tych skryptów i ich użycia można znaleźć w dokumentacji technicznej projektu.

3.5 Przypadki użycia

Główne przypadki użycia systemu TuneScript obejmują:

- **Rejestracja użytkownika:** Nowy użytkownik może utworzyć konto, podając wymagane informacje.
- **Logowanie użytkownika:** Zarejestrowany użytkownik może zalogować się do systemu.
- **Przesyłanie pliku audio:** Użytkownik może przesłać plik audio w formacie MP3 lub WAV do transkrypcji.
- **Generowanie transkrypcji:** System przetwarza przesłany plik audio i generuje transkrypcję MIDI oraz partyturę PDF.
- **Przeglądanie transkrypcji:** Użytkownik może przeglądać swoje oraz publiczne transkrypcje innych użytkowników.
- **Edycja transkrypcji:** Użytkownik może edytować metadane swoich transkrypcji.
- **Wyszukiwanie transkrypcji:** Użytkownik może wyszukiwać transkrypcje na podstawie różnych kryteriów.
- **Ocenianie transkrypcji:** Użytkownik może oceniać transkrypcje innych użytkowników.
- **Dodawanie do ulubionych:** Użytkownik może dodawać transkrypcje do listy ulubionych.
- **Zarządzanie kontem:** Użytkownik może zarządzać ustawieniami swojego konta.

Rozdział 4

Specyfikacja zewnętrzna

4.1 Wymagania sprzętowe i programowe

4.1.1 Wymagania sprzętowe

- Serwer:
 - Procesor: minimum 4 rdzenie, zalecane 8 rdzeni
 - RAM: minimum 8 GB, zalecane 16 GB
 - Dysk: minimum 100 GB przestrzeni dyskowej SSD
- Klient:
 - Dowolne urządzenie z przeglądarką internetową i dostępem do internetu

4.1.2 Wymagania programowe

- Serwer:
 - System operacyjny: Linux (zalecany Ubuntu 20.04 LTS lub nowszy)
 - Docker w wersji 20.10 lub nowszej
 - Docker Compose w wersji 1.29 lub nowszej
- Klient:
 - Nowoczesna przeglądarka internetowa (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)

4.2 Sposób instalacji

Instalacja systemu TuneScript jest prosta i wymaga jedynie kilku kroków:

1. Sklonuj repozytorium projektu:

```
git clone https://github.com/mj300405/TuneScript.git
```

2. Przejdź do katalogu narzędzi deweloperskich projektu:

```
cd TuneScript/devtools
```

3. Uruchom Docker Compose, aby zbudować i uruchomić kontenery (upewnij się, że masz zainstalowany Docker):

```
docker-compose up --build
```

4. Po zakończeniu procesu budowania i uruchamiania, aplikacja będzie dostępna w przeglądarce pod adresem:

```
http://localhost:3000
```

Uwaga: Przed uruchomieniem Docker Compose należy upewnić się, że Docker jest zainstalowany na systemie. Jeśli nie, można pobrać go i zainstalować ze strony oficjalnej: <https://www.docker.com/get-started>.

4.3 Kategorie użytkowników

- **Niezarejestrowani użytkownicy:** Mogą przeglądać publiczne transkrypcje.
- **Zarejestrowani użytkownicy:** Mogą przysyłać pliki audio, generować transkrypcje, zarządzać własnymi transkrypcjami, oceniać i dodawać do ulubionych transkrypcje innych użytkowników.
- **Administratorzy:** Mają pełny dostęp do systemu, w tym do panelu administracyjnego.

4.4 Sposób obsługi

1. Rejestracja i logowanie:

- Użytkownik może zarejestrować się, podając nazwę użytkownika, adres e-mail i hasło.
- Następnie otrzymuje e-mail z linkiem potwierdzającym.
- Po potwierdzeniu, użytkownik może zalogować się do systemu.

2. Przesyłanie pliku audio:

- Zalogowany użytkownik może przesłać plik audio w formacie MP3 lub WAV.
- System weryfikuje format i rozmiar pliku.

3. Generowanie transkrypcji:

- Po przesłaniu pliku, system automatycznie rozpoczyna proces transkrypcji.
- Użytkownik jest informowany o postępie procesu.

4. Zarządzanie transkrypcjami:

- Użytkownik może przeglądać, edytować i usuwać swoje transkrypcje.
- Możliwe jest ustawienie widoczności transkrypcji jako publicznej lub prywatnej.

5. Wyszukiwanie i przeglądanie transkrypcji:

- Użytkownicy mogą wyszukiwać transkrypcje po tytule, kompozytorze i widoczności.
- Możliwe jest przeglądanie publicznych transkrypcji innych użytkowników.

6. Ocenianie i dodawanie do ulubionych:

- Zalogowani użytkownicy mogą oceniać transkrypcje.
- Możliwe jest dodawanie transkrypcji do listy ulubionych.

4.5 Administracja systemem

Administratorzy mają dostęp do panelu administracyjnego Django, który umożliwia:

- Zarządzanie kontami użytkowników
- Przeglądanie i moderowanie transkrypcji
- Monitorowanie aktywności systemu
- Konfigurację ustawień systemu

4.6 Kwestie bezpieczeństwa

- Wszystkie hasła są przechowywane w formie zaszyfrowanej.
- Komunikacja między klientem a serwerem jest szyfrowana za pomocą HTTPS.
- System wykorzystuje tokeny JWT do uwierzytelniania użytkowników.
- Implementacja mechanizmów ochrony przed atakami CSRF i XSS.
- Regularne aktualizacje wszystkich wykorzystywanych bibliotek i zależności.

4.7 Przykład działania

Poniżej przedstawiono przykładowy scenariusz korzystania z systemu TuneScript:

1. Użytkownik rejestruje się w systemie.
2. Po zalogowaniu, użytkownik przesyła plik audio zawierający nagranie fortepianowe.
3. System przetwarza plik i generuje transkrypcję.
4. Użytkownik może przeglądać, edytować i pobierać wygenerowaną transkrypcję.
5. Użytkownik ustawia transkrypcję jako publiczną.
6. Inni użytkownicy mogą przeglądać, oceniać i dodawać transkrypcję do ulubionych.

4.8 Scenariusze korzystania z systemu

Rozdział 5

Specyfikacja wewnętrzna

5.1 Przedstawienie idei

TuneScript to zaawansowana aplikacja webowa, która automatyzuje proces transkrypcji nagrań fortepianowych w formacie MP3 na partytury w formacie PDF. Główna idea projektu opiera się na wykorzystaniu sztucznej inteligencji, a dokładniej głębokich sieci neuronowych, do analizy i interpretacji sygnału audio, a następnie przekształcenia go w notację muzyczną.

Kluczowe aspekty idei TuneScript obejmują:

- Wykorzystanie zaawansowanych technik przetwarzania sygnałów audio do precyzyjnej detekcji nut.
- Implementację modelu głębokiego uczenia do transkrypcji muzyki fortepianowej.
- Integrację procesu transkrypcji z systemem generowania partytur.
- Stworzenie przyjaznego dla użytkownika interfejsu webowego do zarządzania transkrypcjami.

5.2 Architektura systemu

TuneScript został zaprojektowany w oparciu o architekturę mikrousług, co zapewnia elastyczność, skalowalność i łatwość utrzymania. Główne komponenty systemu to:

1. **Frontend:** Implementowany w Next.js i TypeScript, zapewniający responsywny i interaktywny interfejs użytkownika.
2. **Backend API:** Napisany w Django i Graphene, obsługujący żądania od frontendu i zarządzający logiką biznesową.

3. **Serwis transkrypcji:** Odpowiedzialny za przetwarzanie plików audio i generowanie transkrypcji MIDI.
4. **Serwis generowania partytur:** Konwertuje pliki MIDI na partytury PDF.
5. **Baza danych:** PostgreSQL przechowujący dane użytkowników, metadane transkrypcji i inne informacje.
6. **System kolejkowania:** Implementowany z użyciem Celery i Redis, zarządzający asynchronicznymi zadaniami.

5.3 Opis struktur danych i organizacji bazy danych

TuneScript wykorzystuje relacyjną bazę danych PostgreSQL do przechowywania danych. Główne tabele w bazie danych to:

- **Users:** Przechowuje informacje o użytkownikach.
- **Transcriptions:** Zawiera metadane transkrypcji, w tym odniesienia do plików audio, MIDI i PDF.
- **Ratings:** Przechowuje oceny transkrypcji.
- **Favorites:** Zawiera informacje o ulubionych transkrypcjach użytkowników.

5.4 Komponenty, moduły i biblioteki

Główne komponenty i biblioteki wykorzystywane w projekcie TuneScript:

- **Frontend:**
 - Next.js: Framework React do renderowania po stronie serwera
 - TypeScript: Typowany nadzbiór JavaScript
 - Apollo Client: Klient GraphQL do komunikacji z backendem
 - Tailwind CSS: Framework CSS do stylizacji
- **Backend:**
 - Python: Główny język programowania
 - Django: Framework webowy Python
 - Graphene: Biblioteka do implementacji API GraphQL
 - Celery: System kolejkowania zadań

- Redis: Broker wiadomości dla Celery
- PyTorch: Biblioteka do uczenia maszynowego, używana w modelu transkrypcji
- Librosa: Biblioteka do analizy audio
- Flask: Lekki framework webowy
- Transformers: Biblioteka do przetwarzania języka naturalnego
- **Baza danych:**
 - PostgreSQL: System zarządzania relacyjną bazą danych
- **Narzędzia deweloperskie:**
 - Docker i Docker Compose: Do konteneryzacji i orkiestracji usług
 - Poetry: Zarządzanie zależnościami Python
 - ESLint: Narzędzie do analizy statycznej kodu JavaScript/TypeScript
 - Black: Formatter kodu Python
 - isort: Narzędzie do sortowania importów w Pythonie
 - Flake8: Linter kodu Python
- **Narzędzia testowe:**
 - Factory Boy: Biblioteka do generowania obiektów testowych w Pythonie
 - pytest: Framework testowy dla Pythona
 - Jest: Framework testowy dla JavaScript/TypeScript

5.5 Skrypty deweloperskie

Projekt TuneScript zawiera szereg skryptów ułatwiających proces rozwoju i testowania. Są one zdefiniowane w pliku `package.json`:

```
{
  "name": "tunescript-devtools",
  "version": "1.0.0",
  "description": "Development tools for TuneScript",
  "scripts": {
    "start": "docker-compose up -d",
    "stop": "docker-compose down -v",
    "start:persistent": "docker-compose -f docker-compose.persistent.yml up -d",
    "stop:persistent": "docker-compose -f docker-compose.persistent.yml down",
  }
}
```

```

"start:fresh": "docker-compose -f docker-compose.fresh.yml up -d",
"stop:fresh": "docker-compose -f docker-compose.fresh.yml down -v",
"restart": "npm run stop && npm run start",
"restart:persistent": "npm run stop:persistent && npm run start:persistent",
"restart:fresh": "npm run stop:fresh && npm run start:fresh",
"logs": "docker-compose logs -f",
"test": "docker-compose exec frontend npm run test",
"test:watch": "docker-compose exec frontend npm run test:watch",
"test:coverage": "docker-compose exec frontend npm run test:coverage",
"test:backend": "docker-compose exec web poetry run pytest tunescript_app/tests",
"format": "docker-compose exec web /app/format.sh"
}
}

```

Te skrypty umożliwiają łatwe uruchamianie, zatrzymywanie i restartowanie aplikacji w różnych trybach, a także uruchamianie testów i formatowanie kodu.

5.6 Przegląd ważniejszych algorytmów

Kluczowym algorytmem w systemie TuneScript jest model transkrypcji muzycznej oparty na pracy Kong et al. (2020) "High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times". Główne etapy tego algorytmu to:

1. Przetwarzanie wstępne sygnału audio (np. normalizacja, konwersja do spektrogramu).
2. Analiza sygnału przez sieć konwolucyjną w celu wykrycia początków (onset) i końców (offset) nut.
3. Regresja czasów początku i końca każdej nuty.
4. Detekcja użycia pedałów fortepianu.
5. Generowanie transkrypcji MIDI na podstawie wykrytych nut i pedałów.

5.7 Szczegóły implementacji wybranych fragmentów

5.7.1 Implementacja modelu transkrypcji

Model transkrypcji jest implementowany w PyTorch. Oto fragment kodu ilustrujący strukturę głównej sieci neuronowej:

```

class PianoTranscriptionModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 32 * 32, 1024)
        self.fc2 = nn.Linear(1024, 88 * 2) # 88 klawiszy, onset i offset

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)
        x = x.view(-1, 64 * 32 * 32)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

```

5.7.2 Implementacja API GraphQL

Backend wykorzystuje Graphene do implementacji API GraphQL. Oto przykład definicji typu i mutacji dla transkrypcji:

```

class TranscriptionType(DjangoObjectType):
    class Meta:
        model = Transcription
        fields = ("id", "title", "audio_file", "midi_file", "pdf_file")

class CreateTranscription(graphene.Mutation):
    class Arguments:
        title = graphene.String(required=True)
        audio_file = graphene.String(required=True)

    transcription = graphene.Field(TranscriptionType)

    def mutate(self, info, title, audio_file):
        user = info.context.user
        transcription = Transcription.objects.create(
            title=title,
            audio_file=audio_file,

```

```
        user=user
    )
    return CreateTranscription(transcription=transcription)
```

5.8 Diagramy UML

Rozdział 6

Weryfikacja i walidacja

- sposób testowania w ramach pracy (np. odniesienie do modelu V)
- organizacja eksperymentów
- przypadki testowe zakres testowania (pełny/niepełny)
- wykryte i usunięte błędy
- opcjonalnie wyniki badań eksperymentalnych

Tabela 6.1: Nagłówek tabeli jest nad tabelą.

ζ	metoda						
	alg. 1	alg. 2	alg. 3			alg. 4, $\gamma = 2$	
			$\alpha = 1.5$	$\alpha = 2$	$\alpha = 3$	$\beta = 0.1$	$\beta = -0.1$
0	8.3250	1.45305	7.5791	14.8517	20.0028	1.16396	1.1365
5	0.6111	2.27126	6.9952	13.8560	18.6064	1.18659	1.1630
10	11.6126	2.69218	6.2520	12.5202	16.8278	1.23180	1.2045
15	0.5665	2.95046	5.7753	11.4588	15.4837	1.25131	1.2614
20	15.8728	3.07225	5.3071	10.3935	13.8738	1.25307	1.2217
25	0.9791	3.19034	5.4575	9.9533	13.0721	1.27104	1.2640
30	2.0228	3.27474	5.7461	9.7164	12.2637	1.33404	1.3209
35	13.4210	3.36086	6.6735	10.0442	12.0270	1.35385	1.3059
40	13.2226	3.36420	7.7248	10.4495	12.0379	1.34919	1.2768
45	12.8445	3.47436	8.5539	10.8552	12.2773	1.42303	1.4362
50	12.9245	3.58228	9.2702	11.2183	12.3990	1.40922	1.3724

Rozdział 7

Podsumowanie i wnioski

- uzyskane wyniki w świetle postawionych celów i zdefiniowanych wyżej wymagań
- kierunki ewentualnych danych prac (rozbudowa funkcjonalna ...)
- problemy napotkane w trakcie pracy

Bibliografia

- [1] Emmanouil Benetos, Simon Dixon, Dimitrios Giannoulis, Holger Kirchhoff i Anssi Klapuri. „Automatic music transcription: challenges and future directions”. W: *Journal of Intelligent Information Systems* 41.3 (2013), s. 407–434.
- [2] Tian Cheng, Simon Dixon i Matthias Mauch. „Attack/decay analysis of piano sounds using an exponential model”. W: *Journal of the Acoustical Society of America* 139.6 (2016), s. 4332–4341.
- [3] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore i Douglas Eck. „Onsets and frames: Dual-objective piano transcription”. W: *Proceedings of the 19th International Society for Music Information Retrieval Conference*. 2018, s. 50–57.
- [4] Il-Young Jeong i Kyeongok Lee. „Deep Learning for Music”. W: *Proceedings of the 18th International Society for Music Information Retrieval Conference*. 2017, s. 463–469.
- [5] Rainer Kelz, Matthias Dorfer, Filip Korzeniowski, Sebastian Böck, Andreas Arzt i Gerhard Widmer. „On the Potential of Simple Framewise Approaches to Piano Transcription”. W: *Proceedings of the 17th International Society for Music Information Retrieval Conference*. 2016, s. 475–481.
- [6] Qiuqiang Kong, Bochen Li, Xuchen Song, Yuan Wan i Yuxuan Wang. „High-resolution Piano Transcription with Pedals by Regressing Onsets and Offsets Times”. W: *arXiv preprint arXiv:2010.01815* (2020).
- [7] Siddharth Sigtia, Emmanouil Benetos i Simon Dixon. „An end-to-end neural network for polyphonic piano music transcription”. W: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 24.5 (2016), s. 927–939.

Dodatki

Spis skrótów i symboli

DNA kwas deoksyrybonukleinowy (ang. *deoxyribonucleic acid*)

MVC model – widok – kontroler (ang. *model-view-controller*)

N liczebność zbioru danych

μ stopnień przyleżności do zbioru

$\mathbb{E}$ zbiór krawędzi grafu

$\mathcal{L}$ transformata Laplace’a

Źródła

Jeżeli w pracy konieczne jest umieszczenie długich fragmentów kodu źródłowego, należy je przenieść w to miejsce.

```
1 if (_nClusters < 1)
2     throw std::string ("unknown_number_of_clusters");
3 if (_nIterations < 1 and _epsilon < 0)
4     throw std::string ("You should set a maximal number of
        iteration or minimal difference — epsilon.");
5 if (_nIterations > 0 and _epsilon > 0)
6     throw std::string ("Both number of iterations and minimal
        epsilon set — you should set either number of iterations
        or minimal epsilon.");
```

Lista dodatkowych plików, uzupełniających tekst pracy

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- dane testowe,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

Spis rysunków

Spis tabel

6.1	Nagłówek tabeli jest nad tabelą.	26
-----	--	----